Uniwersytet Radomski im. Kazimierza Pułaskiego w Radomiu	
Wydział Transportu, Elektrotechniki i Informatyki	
Techniczne zastosowania sieci neuronowych	
Rok akademicki 2025/2026	
Projekt: Implementacja sieci neuronowej do klasyfikacji znaków P, R, O	
Autorzy:	Ivan Pohodaev Andrii Yevlakh Maksymilian Wieczorek

1. Cel

Tematem projektu jest implementacja i wytrenowanie wielowarstwowej sieci neuronowej typu perceptron „ MLP ” do klasyfikacji ręcznie rysowanych liter: "P", "R" oraz "O".

2. Struktura sieci neuronowej

W projekcie wykorzystano wielowarstwową sieć neuronową typu MLP (Multilayer Perceptron), która została zaprojektowana jako jednowarstwowy perceptron ukryty. Architektura ta opiera się na 256 neuronach wejściowych, co odpowiada liczbie pikseli obrazu o wymiarach 16x16 rozłożonego do postaci wektora jednowymiarowego. Warstwa ukryta składa się ze 128 neuronów wykorzystujących funkcję aktywacji Leaky ReLU, która pozwala na przepływ niewielkich wartości ujemnych i skutecznie zapobiega problemowi zamierających neuronów podczas procesu uczenia. Warstwa wyjściowa zawiera 3 neurony odpowiadające klasyfikowanym literom P, R oraz O, a zastosowana w niej funkcja Softmax pozwala na interpretację wyników w formie rozkładu prawdopodobieństwa. Wagi sieci nie są dobierane przypadkowo, lecz niesocjalizowane losowo z wykorzystaniem metody opartej na ograniczeniu limitu, wyliczanym jako pierwiastek z ilorazu liczby 6 przez sumę wejść i wyjść danej warstwy. Taki sposób inicjalizacji zapewnia stabilną wariancję sygnału w całej strukturze sieci i przyspiesza zbieżność algorytmu treningowego.

$$limit = \sqrt{\frac{6}{nin + nout}}$$

Wzór do wag

Legenda pod wzorem (wyśrodkowana):

- *limit* – wartość ograniczająca zakres losowania wag.
- *nin* – liczba neuronów na wejściu warstwy (skalar).
- *nout* – liczba neuronów na wyjściu warstwy (skalar).

3. Opis algorytmu uczenia

W projekcie wykorzystano algorytm uczenia nadzorowanego oparty na metodzie wstecznej propagacji błędów (czyli Backpropagation) z optymalizacją spadku gradientu. Proces ten polega na iteracyjnym dostosowywaniu macierzy wag **W1** oraz **W2** w celu minimalizacji błędu średnio-kwadratowego (MSE) między predykcją sieci a wartością zadaną. Jako funkcję aktywacji w warstwie ukrytej zastosowano Leaky ReLU, która dopuszcza niewielki przepływ sygnału dla wartości ujemnych (X * 0,01), co zapobiega całkowitemu wyłączaniu się neuronów podczas treningu. Na warstwie wyjściowej użyto funkcji Softmax, która interpretuje wyniki jako prawdopodobieństwo przynależności do jednej z trzech klas: P, R lub O.
Algorytm działa dynamicznie na danych dostarczanych przez użytkownika. Każdy wzorzec wprowadzony do bazy jest automatycznie powielany poprzez augmentację: obroty co 30°

w pełnym zakresie oraz dodanie szum gusowski (wektor n) , co pozwala uzyskać kilkadziesiąt elementów uczących z jednego rysunku i uodparnia sieć na zniekształcenia. Trening odbywa się przez określoną liczbę epok (domyślnie 30), a w każdej z nich zbiór danych jest losowo mieszany , aby zapewnić lepszą zbieżność procesu optymalizacji.

4. Ciąg uczący (CU)

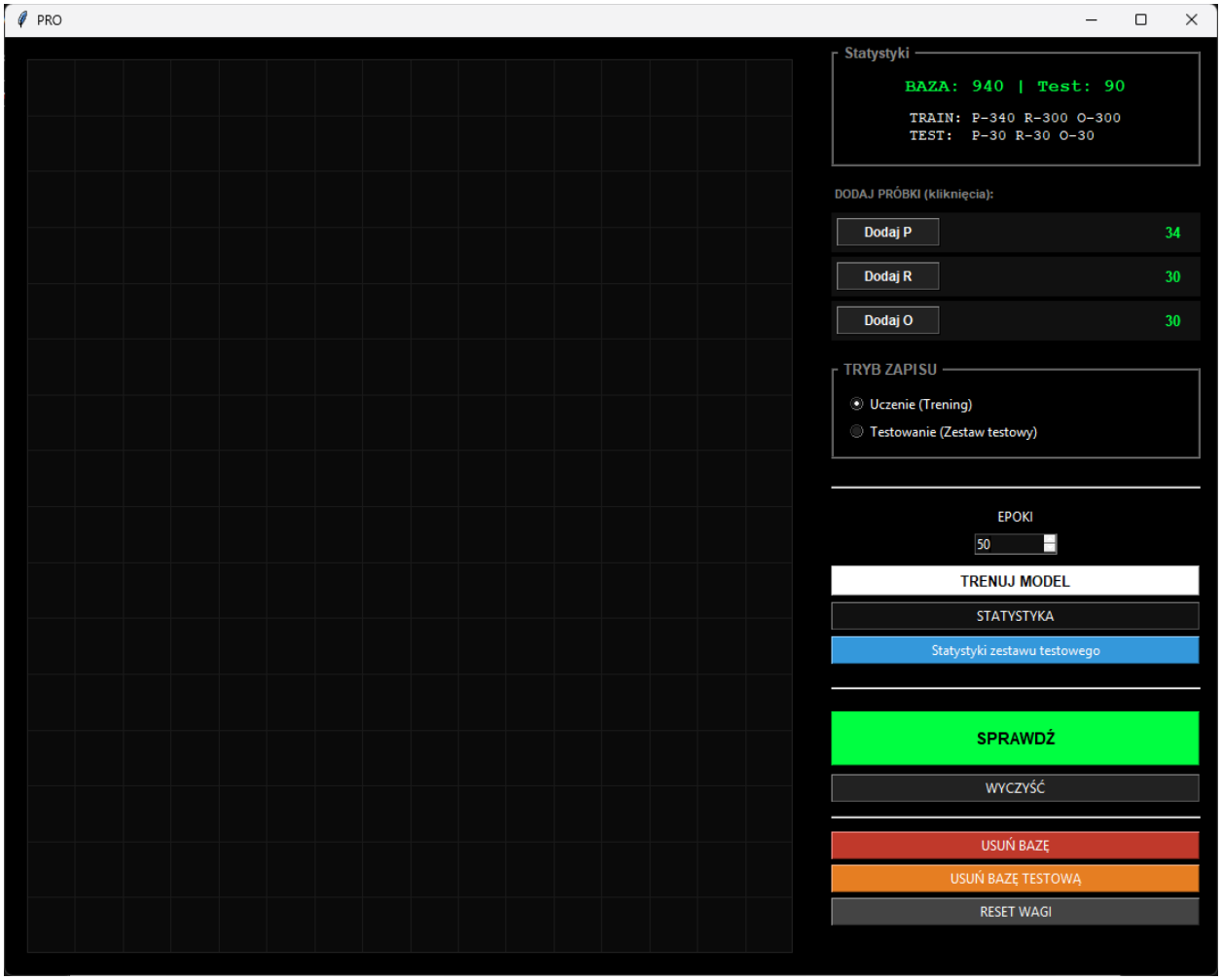
Ciąg uczący został przygotowany w sposób półautomatyczny na podstawie ręcznie wprowadzonych wzorców liter P, R oraz O. Każdy pojedynczy szkic narysowany przez użytkownika na siatce o rozmiarze 16x16 pikseli (co daje 256 wejść do sieci) był poddawany procesowi augmentacji w celu sztucznego zwiększenia bazy danych. Dla każdego wprowadzonego znaku algorytm generował dodatkowe próbki poprzez rotację obrazu co 30 stopni w pełnym zakresie 360 stopni, odbicie lustrzane oraz nałożenie niewielkiego szumu losowego (Gaussian noise). Dzięki temu z jednego ręcznego rysunku uzyskiwano zestaw kilkudziesięciu elementów treningowych. Każdy element ciągu zawiera wartość zadaną w postaci etykiety liczbowej (0 dla P, 1 for R, 2 dla O), zakodowanej w systemie One-Hot podczas obliczeń błędu. Kodowanie danych wejściowych opiera się na binaryzacji – piksele zamalowane przyjmują wartość 1.0, a tło 0.0, po czym macierz jest spłaszczana do wektora jednowymiarowego.

5. Ciąg testowy (CT)

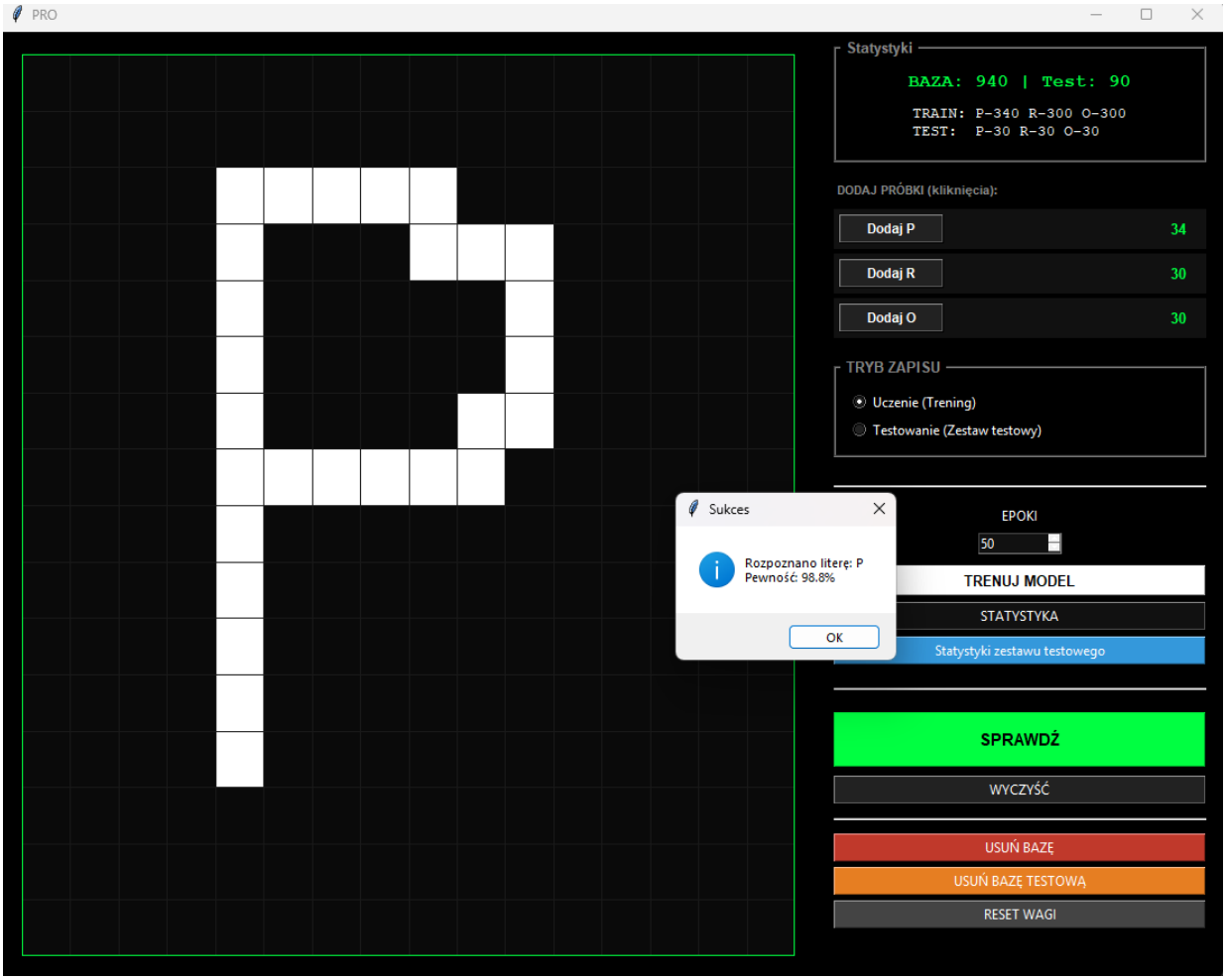
Ciąg testowy składa się z zestawu znaków wprowadzanych przez użytkownika w trybie "Sprawdź", które nie brały udziału w procesie optymalizacji wag. Podobnie jak w przypadku ciągu uczącego, dane testowe reprezentowane są przez wektory o długości 256 elementów, powstałe z przetworzenia rysunku o rozdzielczości 16x16 pikseli. Przed podaniem na wejście sieci, każdy znak testowy przechodzi proces normalizacji: jest wycinany do granic narysowanego kształtu, a następnie skalowany i centrowany wewnątrz obszaru roboczego. Pozwala to na uniezależnienie wyniku klasyfikacji od miejsca, w którym użytkownik narysował literę na ekranie. Skuteczność na tym ciągu jest mierzona poprzez porównanie klasy wskazanej przez sieć (wyjście o najwyższej wartości aktywacji) z faktycznym zamiarem użytkownika, co pozwala określić realną zdolność modelu do rozpoznawania pisma.

6. Implementacja

Aplikacja została przygotowana w języku Python z wykorzystaniem biblioteki Tkinter do stworzenia interfejsu graficznego oraz Matplotlib do wizualizacji wyników. Interfejs składa się z interaktywnego pola rysowniczego o siatce 16x16, na którym użytkownik może kreślić znaki za pomocą myszy. Po prawej stronie panelu znajdują się przyciski sterujące: dodawanie wzorców do bazy (P, R, O), uruchamianie procesu uczenia oraz przycisk "Sprawdź" do klasyfikacji. Obsługa programu jest intuicyjna – po narysowaniu litery użytkownik wybiera odpowiedni przycisk klasy, co automatycznie rozbudowuje zbiór danych o z rotowane i zaszumi one próbki. Po zebraniu odpowiedniej liczby przykładów i przeprowadzeniu treningu, aplikacja staje się gotowa do rozpoznawania nowych rysunków, wyświetlając wynik wraz z procentową pewnością w osobnym oknie.

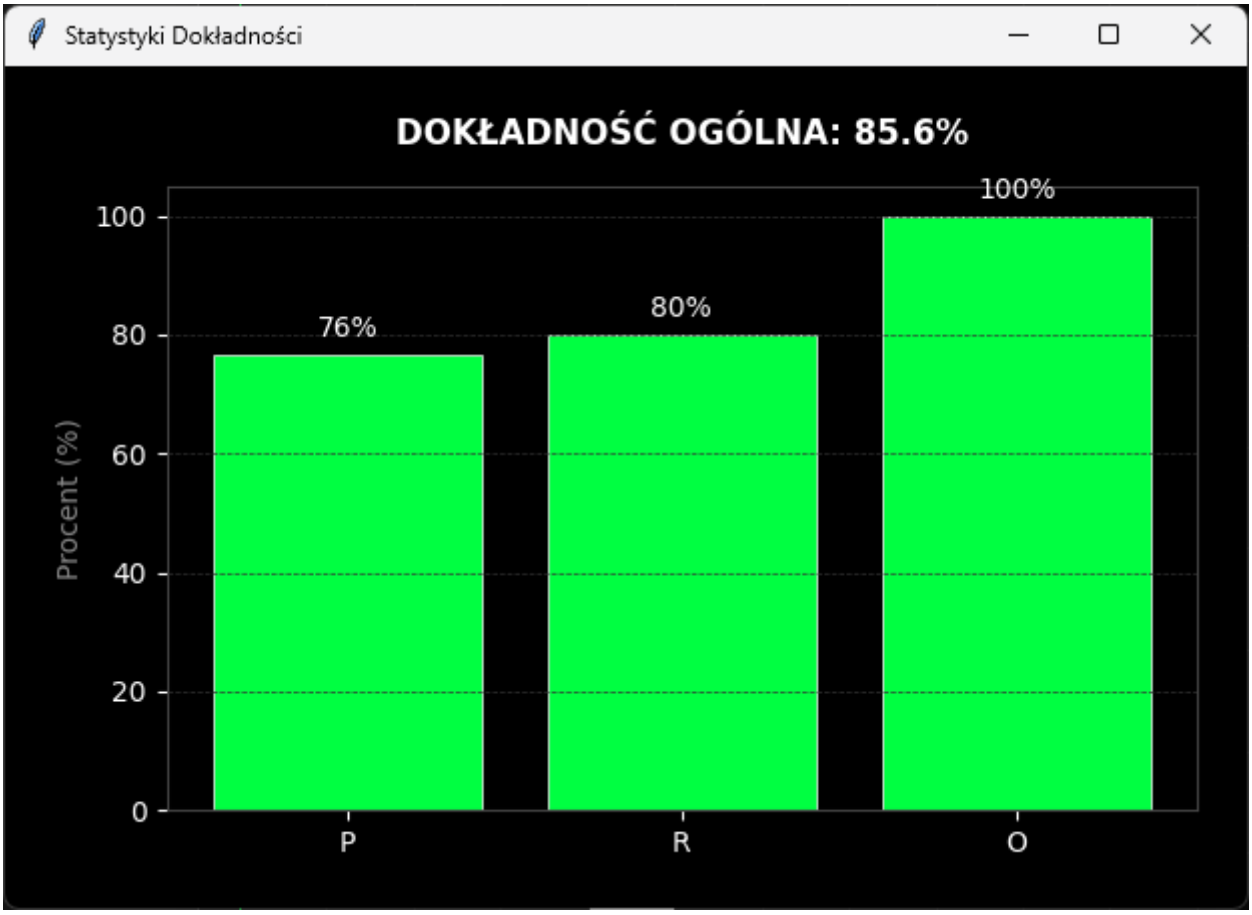


Rys. 1. Główne menu aplikacji z interfejsem graficznym.
Źródło: Opracowanie własne



Rys. 2.Okno wyniku predykcji
Źródło: Opracowanie własne

Rys. 4. Wykres spadku błędu średnio kwadratowego (MSE) w funkcji epok.
Źródło: Opracowanie własne



Rys.4 Statystyki dokładności

Źródło: Opracowanie własne

Instrukcja obsługi

1. Przygotowanie danych (Zbieranie bazy)
   - Rysowanie: Użyj lewego przycisku myszy, aby narysować literę na czarnym polu.
   - Wybór trybu: W sekcji „TRYB ZAPISU" wybierz Uczenie (budowa bazy wiedzy) lub Testowanie (sprawdzanie skuteczności na nowych znakach).
   - Dodawanie: Kliknij Dodaj P, R lub O. Program automatycznie doda warianty litery (obrócone i przesunięte), aby wzmocnić bazę.

2. Trenowanie sieci
   - Ustawienie epok: Wpisz liczbę cykli nauki w polu „EPOKI" (np. 10–50).
   - Start: Kliknij TRENUJ MODEL. Czekaj na komunikat o zakończeniu.
   - Weryfikacja: Przycisk STATYSTYKA wyświetli wykres błędu (Loss) – linia powinna opadać.

3. Rozpoznawanie i testy
   - Klasyfikacja: Narysuj literę i kliknij SPRAWDŹ. System poda rozpoznany znak oraz procentową pewność wyniku.
   - Walidacja: Przycisk Statystyki zestawu testowego pokaże na wykresie słupkowym, jak dobrze model radzi sobie z danymi, których wcześniej „nie widział".

4. Higiena pracy
   - WYCZYŚĆ: Czyści aktualny rysunek.
   - USUŃ BAZĘ: Kasuje wszystkie zebrane próbki z dysku.
   - RESET WAGI: Zeruje pamięć modelu do stanu początkowego.

6.1. Opis najważniejszych fragmentów kodu

Najważniejszym elementem implementacji jest klasa DeepNeuralNetwork, która zawiera kompletną logikę sieci. Kluczowy fragment kodu to metoda train_step, realizująca proces uczenia:

```python
    def train_step(self, X, y_label):
        target = np.zeros((1, 3))
        target[0, y_label] = 1
        output = self.forward(X)
        error = output - target

        # Backpropagation
        d_W2 = self.hidden.T.dot(error)
        error_hidden = error.dot(self.W2.T) * self.leaky_relu_derivative(self.hidden)
        d_W1 = X.T.dot(error_hidden)

        self.W2 -= self.lr * d_W2
        self.W1 -= self.lr * d_W1
        return np.mean(np.square(error))
```

Rys. 5. Algorytm uczenia
Źródło: Opracowanie własne

Opis : Powyższy fragment przedstawia implementację metody train_step, która odpowiada za proces uczenia sieci neuronowej poprzez algorytm wstecznej propagacji błędu. Funkcja wyznacza różnicę między przewidywaniem modelu a oczekiwanym wzorcem (target), a następnie oblicza gradienty wag dla obu warstw sieci z wykorzystaniem pochodnej funkcji aktywacji. Na tej podstawie dokonuje korekty wag **W1** i **W2** przy użyciu współczynnika uczenia lr, dążąc do minimalizacji błędu. Całość procesu kończy się zwróceniem średniego błędu kwadratowego, co pozwala monitorować postępy treningu.

```python
    def __init__(self):
        self.input_size = 256
        self.hidden_size = 128
        self.output_size = 3
        self.lr = 0.001
        self.weights_file = "model_weights.pkl"
        self.history = []
        self.reset_weights()
        self.load_model()

    def reset_weights(self):
        limit = np.sqrt(6 / (self.input_size + self.hidden_size))
        self.W1 = np.random.uniform(-limit, limit, (self.input_size, self.hidden_size))
        self.W2 = np.random.uniform(-limit, limit, (self.hidden_size, self.output_size))
        self.history = []
```

Rys. 6. Funkcje Aktywacji i Propagacja w przód
Źródło: Opracowanie własne

Opis: Powyższy fragment kodu definiuje strukturę sieci neuronowej typu MLP . Parametry input_size (o wartości 256) odpowiadają spłaszczonej macierzy obrazu 16*16, a output_size (o wartości 3) reprezentuje klasyfikowane litery P, R i O. Zastosowano tu profesjonalną inicjalizację wag metodą Xavier , która losuje wartości z zakresu zależnego od liczby połączeń wejściowych i wyjściowych, co zapewnia stabilność procesu uczenia od pierwszej epoki.

```python
    def draw(self, event):
        self.canvas.config(highlightbackground="#222")

        w = self.canvas.winfo_width() / self.grid_size
        h = self.canvas.winfo_height() / self.grid_size
        x, y = int(event.x / w), int(event.y / h)
        if 0 <= x < self.grid_size and 0 <= y < self.grid_size:
            self.drawing_data[y, x] = 1.0
            self.canvas.create_rectangle(
                x * w, y * h, (x + 1) * w, (y + 1) * h, fill="white", outline="#1a1a1a"
            )

```

Rys. 7. Plansza do rysowania
Źródło: Opracowanie własne

Opis: Logika interfejsu graficznego opiera się na mapowaniu współrzędnych kursora myszy na indeksy dwuwymiarowej tablicy NumPy. Każdy 'piksel' narysowany przez użytkownika na komponencie Canvas jest zapisywany w macierzy jako wartość 1.0 . Metoda draw zapewnia natychmiastową informację zwrotną dla użytkownika, jednocześnie przygotowując dane binarne do dalszej obróbki przez sieć neuronową

```python
        for angle in [-12, -6, 6, 12]:
            rot = rotate(processed, angle, reshape=False, order=1)
            self.dataset.append((np.where(rot > 0.3, 1.0, 0.0).flatten(), label))

        for dx, dy in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
            shifted = np.roll(processed, shift=dx, axis=1)
            shifted = np.roll(shifted, shift=dy, axis=0)
            self.dataset.append((shifted.flatten(), label))

        noise = np.random.normal(0, 0.05, processed.shape)
        noised = np.clip(processed + noise, 0, 1)
        self.dataset.append((noised.flatten(), label))
```

Rys. 8 . Augmentacja danych
Źródło: Opracowanie własne

Opis: Ten fragment kodu realizuje proces augmentacji, czyli sztucznego powiększania zbioru treningowego. Z każdego pojedynczego rysunku system generuje kilkanaście dodatkowych próbek poprzez rotację o niewielkie kąty, przesunięcia o jeden piksel w każdym kierunku oraz nałożenie losowego szumu Gaussa. Zabieg ten drastycznie poprawia zdolność generalizacji modelu i zapobiega nadmiernemu dopasowaniu do idealnie narysowanych wzorców.

```python
    def process_image(self, data):
        rows = np.any(data, axis=1)
        cols = np.any(data, axis=0)
        if not np.any(rows):
            return None
        rmin, rmax = np.where(rows)[0][[0, -1]]
        cmin, cmax = np.where(cols)[0][[0, -1]]
        cropped = data[rmin : rmax + 1, cmin : cmax + 1]
        try:
            res = zoom(cropped, (12 / cropped.shape[0], 12 / cropped.shape[1]), order=0)
            final = np.zeros((16, 16))
            final[2:14, 2:14] = res
            return np.where(final > 0.3, 1.0, 0.0)
        except:
            return data
```

Rys. 9. Przetwarzanie obrazu
Źródło: Opracowanie własne

Opis: Metoda process_image odpowiada za normalizację danych wejściowych. Program automatycznie wykrywa krawędzie narysowanego znaku, wycina zbędne tło, a następnie skaluje literę do rozmiaru 12*12 pikseli przy użyciu interpolacji zoom. Dzięki osadzeniu tak przygotowanego znaku w stałej ramce 16*16, sieć staje się odporna na różnice w wielkości rysunku oraz jego położeniu na polu roboczym.

```python
    def save_model(self):
        with open(self.weights_file, "wb") as f:
            pickle.dump({"W1": self.W1, "W2": self.W2, "history": self.history}, f)
```

Rys. 10.Zapis Binarny
Źródło: Opracowanie własne

Opis: Do trwałego przechowywania parametrów modelu oraz zgromadzonej bazy danych wykorzystano moduł pickle. Dane są zapisywane w formacie binarnym (.pkl), co pozwala na zachowanie pełnej precyzji wag sieci oraz struktury obiektów bez konieczności ich konwersji na formaty tekstowe. Dzięki temu model może być trenowany wielokrotnie, a postępy nauki są automatycznie wczytywane przy każdym uruchomieniu aplikacji.

```python
    def leaky_relu(self, x):
        return np.where(x > 0, x, x * 0.01)

    def leaky_relu_derivative(self, x):
        return np.where(x > 0, 1, 0.01)

    def softmax(self, x, T=1.0):
        ex = np.exp((x - np.max(x)) / T)
        return ex / np.sum(ex, axis=1, keepdims=True)

```

Rys. 11. Funkcja Aktywacyjna
Źródło: Opracowanie własne

Opis: W warstwie ukrytej zastosowano funkcję aktywacji Leaky ReLU, która pozwala na przepływ minimalnych wartości ujemnych, co zapobiega całkowitemu wyłączaniu się neuronów podczas treningu. Warstwa wyjściowa korzysta z funkcji Softmax, która przekształca surowe wyniki sieci na wartości prawdopodobieństwa. Suma wyników dla wszystkich klas wynosi 1.0, co pozwala precyzyjnie określić pewność modelu podczas predykcji

7. Proces uczenia

Proces uczenia sieci neuronowej został przeprowadzony w trybie iteracyjnym, w którym podstawową jednostką miary czasu trwania sesji była epoka, definiowana jako jednokrotne przejście algorytmu przez cały dostępny zbiór danych treningowych. Liczba epok jest parametrem konfigurowalnym przez użytkownika w interfejsie graficznym aplikacji, co pozwala na elastyczne dostosowanie czasu treningu do bieżących potrzeb oraz wielkości zgromadzonej bazy próbek. Współczynnik uczenia został ustalony na stałym poziomie 0.001, co zapewnia stabilną aktualizację wag i minimalizuje ryzyko wystąpienia oscylacji wokół minimum funkcji straty. W celu poprawy zdolności generalizacji modelu, przed rozpoczęciem każdej epoki zbiór treningowy jest poddawany losowemu mieszaniu, co zapobiega zapamiętywaniu przez sieć sekwencji wprowadzanych danych. Program na bieżąco monitoruje postępy procesu poprzez obliczanie średniego błędu kwadratowego dla każdej iteracji, a uzyskana historia strat jest archiwizowana w celu późniejszej wizualizacji na wykresie. Proces uczenia kończy się po wykonaniu zadanej liczby cykli, a finalne wartości wag są automatycznie zapisywane w pliku binarnym, co umożliwia ich późniejsze odtworzenie bez konieczności powtórnego trenowania modelu. Istotnym elementem wzmacniającym ten proces jest automatyczna augmentacja danych podczas dodawania próbek, dzięki której sieć uczy się rozpoznawać litery mimo ich przesunięć, rotacji czy obecności szumu cyfrowego.

8. Testowanie

Proces testowania zrealizowano przy użyciu niezależnego zbioru danych zapisanego w osobnym pliku binarnym, co pozwoliło na obiektywną ocenę skuteczności modelu na danych, których nie widział on w fazie uczenia. System klasyfikacji poddano weryfikacji poprzez obliczenie ogólnej dokładności , zdefiniowanej jako stosunek liczby poprawnych wskazań do sumy wszystkich próbek testowych. Wyniki te zestawiono w formie graficznego wykresu słupkowego, który prezentuje skuteczność rozpoznawania z rozbiciem na poszczególne klasy liter P, R oraz O, co ułatwia identyfikację ewentualnych błędów wynikających z podobieństwa wizualnego znaków. Dodatkowym elementem procedury testowej jest wbudowany mechanizm kontroli pewności, który analizuje rozkład prawdopodobieństwa na wyjściu sieci; w przypadku, gdy różnica między dwiema najbardziej prawdopodobnymi klasami jest zbyt niska lub najwyższy wynik nie przekracza określonego progu, system zgłasza niepewność klasyfikacji. Takie podejście pozwala na precyzyjne określenie granic możliwości modelu oraz wpływu parametrów takich jak liczba epok czy jakość rysunku na ostateczną poprawność działania programu.

9. Wnioski

Przeprowadzone badania nad działaniem zaimplementowanej sieci neuronowej typu MLP wykazały, że model skutecznie radzi sobie z klasyfikacją odręcznie rysowanych liter P, R oraz O, osiągając wysoką powtarzalność wyników przy odpowiednio dobranym zbiorze treningowym. Kluczowym elementem wpływającym na stabilność pracy sieci okazał się proces wstępnego przetwarzania obrazu, w tym kadrowanie i skalowanie, który zniwelował różnice wynikające z wielkości oraz położenia rysunku na kanwie. Zastosowanie mechanizmu augmentacji danych, polegającego na automatycznym generowaniu rotacji i przesunięć każdej wprowadzonej próbki, znacząco poprawiło zdolność generalizacji modelu, pozwalając na poprawną identyfikację znaków nawet przy ich znacznym pochyleniu. Zaobserwowano, że funkcja aktywacji Leaky ReLU skutecznie zapobiegała problemowi zamierania neuronów w warstwie ukrytej, co w połączeniu z odpowiednio niskim współczynnikiem uczenia (0.001) pozwoliło na płynną minimalizację błędu średniokwadratowego w kolejnych epokach. System kontroli pewności (progi prawdopodobieństwa i różnicy między klasami) okazał się użytecznym narzędziem diagnostycznym, poprawnie wskazującym sytuacje, w których litery były narysowane w sposób niejednoznaczny, co zapobiegało błędnym klasyfikacjom „na siłę”. Podsumowując, autorska implementacja sieci bez wykorzystania gotowych bibliotek uczenia

maszynowego potwierdziła swoją skuteczność w zadaniu rozpoznawania prostych wzorców graficznych, stanowiąc solidną bazę do dalszej rozbudowy o kolejne klasy znaków.

10. Bibliografia

[1] Geron A., *Uczenie maszynowe z użyciem Scikit-Learn i TensorFlow*, Helion, Gliwice 2020.
[2] Goodfellow I., Bengio Y., Courville A., *Deep Learning. Systemy uczące się*, Wydawnictwo Naukowe PWN, Warszawa 2018.
[3] *NumPy Documentation*, https://numpy.org/doc/, data ostatniego dostępu: 26.03.2026.
[4] Rutkowski L., *Metody i techniki sztucznej inteligencji*, Wydawnictwo Naukowe PWN, Warszawa 2012.
[5] Tadeusiewicz R., *Sieci neuronowe*, Wydawnictwo Akademickie i Profesjonalne, Warszawa 2015.
[6] *Tkinter GUI library*, https://docs.python.org/3/library/tkinter.html, data ostatniego dostępu: 26.03.2026.